Security Audit Report

Project: Ratatoskr (backend — FastAPI + Telegram bot + Mobile API + MCP server + Taskiq worker) **Date**: 2026-06-27 **Auditor**: Claude Security Audit **Frameworks**: OWASP Top 10:2025 + NIST CSF 2.0 (+ CWE, SANS Top 25, ASVS 5.0, PCI DSS 4.0.1, MITRE ATT&CK, SOC 2, ISO 27001:2022) **Mode**: full (deep) — Phases 1-5, white-box + gray-box + hotspots + smells

---

Executive Summary

Metric	Count
Critical	0
High	10
Medium	15
Low	12
Informational	4
Gray-box findings	5
Security hotspots	8
Code smells	6
Total findings	**60**

Overall Risk Assessment: Ratatoskr has a notably strong security foundation — fail-closed authentication (deny-by-default when `ALLOWED_USER_IDS` is empty), constant-time secret comparison (`hmac.compare_digest`), argon2id for password credentials, Fernet at-rest encryption of tokens, a custom IP-pinning SSRF transport (`app/security/ssrf.py`) applied to all direct outbound HTTP, parameterized SQL throughout (zero raw-SQL injection found), `shell=False` subprocess usage everywhere, non-root containers with `no-new-privileges`, loopback-bound ports, and consistent log redaction of `Authorization` headers. **No CRITICAL issues and no traditional injection (SQLi/command/SSTI/deserialization-from-request) were found.** The residual risk concentrates in four themes: (1) rate-limiting that silently degrades or fails open when Redis is unavailable and is ineffective behind a reverse proxy; (2) missing HTTP security response headers and host-validation middleware on an app that serves a Telegram-WebApp auth SPA; (3) software-supply-chain hygiene in CI/Docker (mutable action tags, dependency-confusion index strategy, a disabled dependency-review gate, an unpinned upstream-`main` sidecar build); and (4) LLM-pipeline prompt-injection isolation that is advisory-only on some paths plus a second-order RAG-poisoning path. Most issues are materially mitigated by the documented single-tenant / owner-only deployment model, but several become serious the moment the service is exposed behind a proxy, scaled to multiple workers, or expanded to multi-tenant.

OWASP Top 10:2025 Coverage

OWASP ID	Category	Findings	Status
A01:2025	Broken Access Control (incl. SSRF)	4	Needs Attention
A02:2025	Security Misconfiguration	7	Needs Attention
A03:2025	Software Supply Chain Failures	5	Needs Attention
A04:2025	Cryptographic Failures	7	Needs Attention
A05:2025	Injection (incl. prompt injection, XSS)	3	Needs Attention

OWASP ID	Category	Findings	Status
A06:2025	Insecure Design	5	Needs Attention
A07:2025	Authentication Failures	6	Needs Attention
A08:2025	Software or Data Integrity Failures	3	Needs Attention
A09:2025	Security Logging and Alerting Failures	5	Needs Attention
A10:2025	Mishandling of Exceptional Conditions	4	Needs Attention

Traditional injection (SQLi, OS command, SSTI, untrusted deserialization-from-request), CSRF (no cookie-auth state-changing browser flows; JWT bearer + Telegram HMAC), and hardcoded-production-secret leakage are **clean** — see "Confirmed-Clean Areas".

NIST CSF 2.0 Coverage

Function	Categories	Findings	Status
GV (Govern)	GV.SC, GV.RM, GV.PO	6	Needs Attention
ID (Identify)	ID.AM, ID.RA	3	Partial
PR (Protect)	PR.AA, PR.DS, PR.PS, PR.IR	30	Needs Attention
DE (Detect)	DE.CM, DE.AE	9	Needs Attention
RS (Respond)	RS.MA, RS.AN, RS.CO, RS.MI	2	Partial
RC (Recover)	RC.RP, RC.CO	0	Out of audit scope

Compliance Coverage

Framework	Coverage	Details
CWE	23 unique CWEs	CWE-918, CWE-352(n/a), CWE-348, CWE-693, CWE-1188, CWE-1392, CWE-916, CWE-502, CWE-494, CWE-829, CWE-74, CWE-77, CWE-20, CWE-209, CWE-532, CWE-359, CWE-778, CWE-390, CWE-703, CWE-400, CWE-613, CWE-598, CWE-320
SANS/CWE Top 25	6/25 matched	#16 (CWE-502), #19 (CWE-918), #20 (CWE-494 supply chain), #21 (CWE-476-adjacent), #24 (CWE-400), #14 (CWE-287)
OWASP ASVS 5.0	9/14 chapters	V1, V2, V4, V5, V6, V7, V8, V11, V14

Framework	Coverage	Details
PCI DSS 4.0.1	Relevant: 6.2-6.4, 8.2-8.6, 10.2-10.7, 2.2	No cardholder data in scope; mapped for completeness
MITRE ATT&CK	7 techniques	T1090, T1190, T1110, T1195/T1195.002, T1078, T1499/T1498, T1592
SOC 2	8 criteria	CC6.1, CC6.3, CC6.6, CC6.7, CC6.8, CC7.1, CC7.2, A1.1
ISO 27001:2022	11 controls	A.8.5, A.8.6, A.8.8, A.8.9, A.8.11, A.8.16, A.8.24, A.8.26, A.8.27, A.8.28, A.12.5.1

Critical & High Findings

No CRITICAL findings.

[HIGH-001] Rate limiter degrades to per-process counters / fails open on Redis unavailability

- **Severity**: HIGH
- **OWASP**: A06:2025 (Insecure Design) / A07:2025 (Authentication Failures)
- **CWE**: CWE-799 (Improper Control of Interaction Frequency) + CWE-703 (Improper Handling of Exceptional Conditions)
- **NIST CSF**: PR.AA, DE.CM
- **Compliance**: SANS Top 25 #16-adjacent | ASVS V4.10.2, V11.1.6 | PCI DSS 6.4.3 | ATT&CK T1498/T1110 | SOC2 CC6.1 | ISO A.8.6
- **Location**: `app/api/middleware.py:685-722`; `app/security/rate_limiter.py:302-316`
- **Attack Vector**: `cfg.redis.required` defaults to `false`. When Redis is down (crash, failover, or attacker-induced connection flood) the Mobile-API limiter silently falls through to a module-level in-memory `LocalRateLimiter`. In a multi-worker uvicorn (`--workers N`) or multi-pod deployment each process keeps an independent counter, so the effective cap becomes `N × limit`. Separately, the Telegram `RedisUserRateLimiter.check_and_record()` returns `(True, None)` — explicitly allowing the request — on any Redis `TimeoutError`. An attacker who can degrade Redis nullifies brute-force protection on `/v1/auth/secret-login` and `/v1/auth/credentials-login`.
- **Impact**: Authentication brute-force / credential-stuffing protection is silently void during Redis degradation and weakened in any multi-worker deployment. The degradation is unobservable after the one-time warning (see MEDIUM-014).
- **Vulnerable Code**:

```
if redis_client is None:
    _log_redis_unavailable_once(cfg, ...)
    if cfg.redis.required:          # default False
        return JSONResponse(503, ...)
    return await _handle_local_rate_limit(...)   # per-process fallback
# rate_limiter.py:
except TimeoutError:
    logger.warning("redis_rate_limit_timeout", ...)
    return True, None               # FAIL-OPEN
```

- **Remediation**: Make `REDIS_REQUIRED=true` the default for any non-dev `APP_ENV`, or fail closed (deny / 503) on Redis unavailability specifically for auth-bucket paths. Replace the Telegram limiter's fail-open `TimeoutError` branch with a fail-closed default for sensitive operations. Use a shared store (Redis) as the single source of truth for auth rate limits and never per-process memory in production.

[HIGH-002] Per-IP rate limiting ineffective behind a reverse proxy (X-Forwarded-For ignored)

- **Severity**: HIGH
- **OWASP**: A07:2025 (Authentication Failures) / A06:2025
- **CWE**: CWE-348 (Use of Less Trusted Source for IP Address)
- **NIST CSF**: PR.AA
- **Compliance**: ASVS V4.10.3 | PCI DSS 6.4.1 | ATT&CK T1110/T1499 | SOC2 CC6.1 | ISO A.8.6
- **Location**: `app/api/middleware.py:183-187, 666-673`
- **Attack Vector**: `_get_client_ip()` reads only `request.client.host`. Any internet-facing deployment sits behind nginx / Cloudflare / a load balancer, so every request carries the *proxy's* IP. All clients collapse into one shared rate-limit bucket: per-IP brute-force limits become meaningless (attacker rotates real source IPs freely), while a single busy legitimate client can exhaust the shared bucket for everyone.
- **Impact**: IP-scoped auth rate limiting provides effectively zero protection in production; aggregate false-positive lockouts for legitimate users.
- **Vulnerable Code**:

```python
def _get_client_ip(request: Request) -> str:
    if request.client and request.client.host:
        return request.client.host   # proxy IP in production
    return "unknown"
```

- **Remediation**: Parse `X-Forwarded-For` / `Forwarded` **only** from a configured set of trusted proxy hops (never blindly trust the header), e.g. via Starlette `ProxyHeadersMiddleware` / uvicorn `--forwarded-allow-ips`, and use the left-most untrusted address as the rate-limit key. Document the required proxy configuration.

[HIGH-003] Missing HTTP security response headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy)

- **Severity**: HIGH
- **OWASP**: A02:2025 (Security Misconfiguration)
- **CWE**: CWE-693 (Protection Mechanism Failure) / CWE-1021 (Improper Restriction of Rendered UI Layers)
- **NIST CSF**: PR.PS, PR.DS
- **Compliance**: ASVS V14.4.1-V14.4.7 | PCI DSS 6.4.1 | SOC2 CC6.6 | ISO A.14.1.2
- **Location**: `app/api/main.py:229-254` (only CORS, rate-limit, webapp-auth, correlation-id middleware registered)
- **Attack Vector**: The backend serves the SPA at `/` and `/{path:path}` and hosts the Telegram-WebApp auth flow. With no `X-Frame-Options`/`frame-ancestors`, the auth UI can be framed for clickjacking; with no `X-Content-Type-Options: nosniff`, JSON/asset responses are MIME-sniffable; with no HSTS, TLS can be stripped by a network MitM; with no CSP, any reflected/LLM-sourced HTML escape gap escalates to full XSS.
- **Impact**: Clickjacking of the WebApp authentication flow, MIME-confusion XSS, TLS downgrade, and loss of defense-in-depth against XSS on a page that renders LLM-generated content.

- **Remediation**: Add a security-headers middleware setting `Content-Security-Policy` (default-src 'self', frame-ancestors 'none'), `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, and a restrictive `Permissions-Policy`. Pair with `TrustedHostMiddleware` (see MEDIUM-011).

[HIGH-004] Webwright sidecar image built from unpinned upstream `main` (download of code without integrity check)

- **Severity**: HIGH
- **OWASP**: A08:2025 (Software or Data Integrity Failures)
- **CWE**: CWE-494 (Download of Code Without Integrity Check)
- **NIST CSF**: GV.SC, PR.DS
- **Compliance**: SANS Top 25 #20 | ASVS V10.3.2 | SOC2 CC6.8 | ISO A.12.5.1 | ATT&CK T1195.001
- **Location**: `ops/docker/webwright/Dockerfile:14`
- **Attack Vector**: `ARG WEBWRIGHT_REF=main` followed by `git clone --depth 1 … && git checkout FETCH_HEAD`. Any commit pushed to `microsoft/Webwright main` is pulled into the next `docker compose build` of the `with-webwright` profile and runs as the sidecar with an OpenRouter API key in its environment and write access to `/data/webwright`.
- **Impact**: Build-time RCE in the sidecar, LLM-API-key exfiltration, scrape/trajectory poisoning. Blast radius limited because the `with-webwright` profile is opt-in and default-off, but integrity is unverifiable when it is enabled.
- **Remediation**: Pin `WEBWRIGHT_REF` to a specific commit SHA (or a vendored fork you control) and verify the checkout. Apply the same to any other sidecar built from a moving ref.

[HIGH-005] GitHub Actions pinned to mutable tags instead of immutable commit SHAs

- **Severity**: HIGH
- **OWASP**: A03:2025 (Software Supply Chain Failures)
- **CWE**: CWE-494 (Download of Code Without Integrity Check)
- **NIST CSF**: GV.SC, PR.DS
- **Compliance**: SANS Top 25 #20 | ASVS V10.3.2 | SOC2 CC6.8 | ISO A.12.5.1 | ATT&CK T1195.001
- **Location**: `.github/workflows/ci.yml:33,51,116,135,193,779`; `release.yml:18,31,49,55,63,75`; `regenerate-lockfiles.yml:29,37,99`; `update-locks.yml:29,38,145,164`
- **Attack Vector**: All third-party actions use mutable refs (`actions/checkout@v7`, `astral-sh/setup-uv@v7`, `docker/build-push-action@v7`, `gitleaks/gitleaks-action@v3`, `google/osv-scanner-action@v2.3.8`, `softprops/action-gh-release@v3`). A compromised or hijacked publisher can re-point a tag; the next CI run executes attacker code with repository write access and access to all secrets (`SAFETY_API_KEY`, `ACTIONS_PR_TOKEN`, `GITHUB_TOKEN`) plus GHCR push rights.
- **Impact**: Full CI/CD compromise: secret exfiltration and poisoned Docker images published under legitimate release tags.
- **Remediation**: Pin every third-party action to a full commit SHA (`uses: actions/checkout@<sha> # v7`). Enable Dependabot updates for actions to keep SHAs current. Constrain `GITHUB_TOKEN` permissions per-job with least privilege.

[HIGH-006] `UV_INDEX_STRATEGY=unsafe-best-match` enables dependency confusion at lockfile-generation time

- **Severity**: HIGH
- **OWASP**: A03:2025 (Software Supply Chain Failures)
- **CWE**: CWE-829 (Inclusion of Functionality from Untrusted Control Sphere) / CWE-427
- **NIST CSF**: GV.SC, ID.RA
- **Compliance**: SANS Top 25 #20 | ASVS V14.2.1 | PCI DSS 6.3.2 | SOC2 CC6.8 | ISO A.8.8 | ATT&CK T1195.002
- **Location**: `.github/workflows/ci.yml:22`; `regenerate-lockfiles.yml:20`; `update-locks.yml:23`

- **Attack Vector**: `unsafe-best-match` resolves each package to the highest version across *all* configured indexes. An attacker who publishes a public PyPI package whose name matches a private/Safety-index package, at a higher version, gets it silently selected and committed into `uv.lock` / `requirements*.txt` before any scan runs.
- **Impact**: Supply-chain compromise of the entire Python dependency graph at lock-regeneration time; every container built from the poisoned lock runs attacker code. (Partly mitigated by the workflows' Safety-index reachability guard, but the index strategy itself remains unsafe.)
- **Remediation**: Use `UV_INDEX_STRATEGY=first-index` (or `unsafe-first-match`) and explicitly scope each package to its intended index; never let a public index override a private one by version.

[HIGH-007] Dependency-review gate disabled via `continue-on-error: true`

- **Severity**: HIGH
- **OWASP**: A03:2025 (Software Supply Chain Failures)
- **CWE**: CWE-693 (Protection Mechanism Failure)
- **NIST CSF**: GV.SC, ID.RA
- **Compliance**: ASVS V14.2.2 | PCI DSS 6.3.2 | SOC2 CC7.1 | ISO A.8.8
- **Location**: `.github/workflows/dependency-review.yml:33`
- **Attack Vector**: The `dependency-review-action` is configured `fail-on-severity: high` but also `continue-on-error: true`, so a PR introducing a dependency with a known HIGH/CRITICAL CVE passes the job regardless. The bypass is documented as temporary "until Dependency Graph is enabled" with no enforcement date.
- **Impact**: Known-vulnerable dependencies can be merged to `main` with no automated block.
- **Remediation**: Enable the GitHub Dependency Graph for the repo and remove `continue-on-error`, or replace with an equivalent blocking scan (the repo already runs pip-audit/Safety/OSV — make at least one of them a required, blocking status check).

[HIGH-008] Indirect prompt injection: untrusted content reaches the LLM without structural isolation; detection is advisory-only and trivially bypassed

- **Severity**: HIGH
- **OWASP**: A05:2025 (Injection) / LLM01:2025 (Prompt Injection)
- **CWE**: CWE-74 / CWE-77 (Improper Neutralization) + CWE-20 (Improper Input Validation)
- **NIST CSF**: PR.DS, DE.CM
- **Compliance**: OWASP LLM Top 10 #1 | ASVS V5.2 | ATT&CK T1059 | SOC2 CC6.1 | ISO A.14.2.5
- **Location**: `app/agents/web_search_agent.py:226`; `app/application/services/summarization/graph_llm.py:31` detection at `app/core/content_cleaner.py:24-56`, `app/application/services/summarization/graph_prompt.py`
- **Attack Vector**: The main summarize path correctly wraps scraped content in `<untrusted_source_content>` tags with a SECURITY-BOUNDARY notice, but the web-search analysis path and the enrichment two-pass path concatenate page content into the user role with only plain-text markers (or none). Separately, prompt-injection detection compiles 4 English-only `\b`-anchored regexes whose result merely flips a `quality.prompt_injection_suspected` metadata flag — it does not block the call or alter the prompt. Bypass via homoglyphs, punctuation insertion, paraphrase, or non-English ( ) is trivial, and the summary then reports `prompt_injection_suspected=false`, giving false assurance.
- **Impact**: Attacker-controlled scraped/RAG/bookmark content can redirect the LLM (e.g., steer web-search queries, override enrichment fields, attempt system-prompt exfiltration) with no reliable detection.
- **Remediation**: Apply the same `<untrusted_source_content>` boundary + SECURITY notice on every path that sends scraped or retrieved content to a model (web-search, enrichment, RAG). Treat detection as defense-in-depth only; do not rely on it. Consider an LLM-based or multilingual classifier and structurally separate instructions from data in every call.

[HIGH-009] Global exception handler returns raw exception strings when `LOG_LEVEL=DEBUG`

- **Severity**: HIGH
- **OWASP**: A10:2025 (Mishandling of Exceptional Conditions)
- **CWE**: CWE-209 (Generation of Error Message Containing Sensitive Information)
- **NIST CSF**: DE.AE, PR.DS
- **Compliance**: SANS Top 25 #3-adjacent | ASVS V7.4.1 | SOC2 CC6.1 | ISO A.14.2.1 | ATT&CK T1592
- **Location**: `app/api/error_handlers.py:144-147`
- **Attack Vector**: The production/debug switch is keyed on `config.runtime.log_level == "DEBUG"` rather than `APP_ENV`. Any operator who raises the log level to DEBUG for troubleshooting causes all unhandled 500s to return `str(exc)` verbatim to the client — SQLAlchemy errors leak DB host/schema, httpx errors leak internal service URLs.
- **Impact**: Internal-architecture disclosure to API clients whenever DEBUG logging is enabled, a routine and easily-forgotten operational state.
- **Remediation**: Decouple verbose error responses from log level. Return generic messages plus the `Error ID: <correlation_id>` in all environments; only expose `str(exc)` when `APP_ENV` is explicitly a non-production value, never as a side effect of log verbosity.

[HIGH-010] RAG grounding block built from stored summary fields → second-order (stored) prompt injection into the system prompt

- **Severity**: HIGH (MEDIUM in a strictly single-owner deployment)
- **OWASP**: A05:2025 (Injection) / A08:2025 (Data Integrity) / LLM01:2025
- **CWE**: CWE-74 (Improper Neutralization of Special Elements in Output Used by a Downstream Component)
- **NIST CSF**: PR.DS, GV.SC
- **Compliance**: OWASP LLM Top 10 #1/#5 | ASVS V5.3 | ATT&CK T1195 | SOC2 CC6.6 | ISO A.14.2.2
- **Location**: `app/application/graphs/summarize/nodes/build_prompt.py:93-94`; `app/application/graphs/summa`
- **Attack Vector**: When `SUMMARIZE_RAG_ENABLED`, the grounding block is assembled from prior summaries' `title/tldr` Qdrant payload fields and concatenated directly into the **system** role: `system_prompt = f"{system_prompt.rstrip()}\n\n{block}"`. The `_GROUNDING_GUARD` notice is advisory only. If any earlier injection persisted attacker text into a stored summary's `title/tldr` (280 chars is enough), it resurfaces verbatim in the system prompt of every future request that retrieves it.
- **Impact**: A single persisted injection propagates into subsequent requests' system prompts. Lower in a strictly single-user deployment, but a genuine self-poisoning / cross-context path; becomes cross-user the moment RAG scope spans users.
- **Remediation**: Place grounding/retrieved content in a clearly-delimited user-role block (not the system prompt), wrapped in the untrusted-content boundary. Sanitize/escape retrieved payload fields; never promote retrieved data to system-instruction trust level.

Medium Findings

[MEDIUM-001] SSRF DNS-rebinding TOCTOU in scraper providers and git clone (independent re-resolution at connect time)

- **Severity**: MEDIUM
- **OWASP**: A01:2025 (Broken Access Control — SSRF)
- **CWE**: CWE-918 (Server-Side Request Forgery)
- **NIST CSF**: PR.AA, PR.DS
- **Compliance**: SANS Top 25 #19 | ASVS V12.6 | SOC2 CC6.6 | ISO A.13.1.3 | ATT&CK T1090

- **Location**: app/adapters/content/scraper/crawlee_provider.py:63-70,200; app/adapters/content/scraper/p (acknowledged in code); app/adapters/git_backup/mirror_service.py:808-827
- **Attack Vector**: These three paths validate the target host (`reject_unsafe_target_url` / `is_url_safe` / `assert_resolved_public_host`) and then hand the URL to a component that performs its own independent DNS resolution at connect time (crawlee's httpx client, Chromium's net stack, the `git` subprocess). A TTL-0 record that answers "public" during validation and "169.254.169.254 / 10.x" at connect bypasses the check. The direct-httpx paths are NOT affected because `SafeAsyncTransport` rewrites the URL to the pinned resolved IP (see hotspot).
- **Impact**: Read cloud-metadata credentials or reach internal services via the affected providers; git path is limited to TCP probing / partial error-body leakage.
- **Remediation**: Route crawlee/Playwright through a forced egress proxy that enforces the IP allowlist, or pin DNS at the component boundary (resolve once, pass the IP + `Host` header / `--resolve`). For git, resolve and connect to a pinned IP, or run clones inside a network namespace with egress filtering.

[MEDIUM-002] LangGraph checkpoint pickle fallback re-enabled by `LANGGRAPH_STRICT_MSGPACK=false`

- **Severity**: MEDIUM
- **OWASP**: A08:2025 (Software or Data Integrity Failures)
- **CWE**: CWE-502 (Deserialization of Untrusted Data)
- **NIST CSF**: PR.DS, PR.PS
- **Compliance**: SANS Top 25 #16 | ASVS V1.14.6 | SOC2 CC6.7 | ISO A.14.2.1
- **Location**: app/infrastructure/checkpointing/runtime.py:100; app/config/langgraph.py:36-44
- **Attack Vector**: Default is safe (`strict_msgpack=True`). If an operator sets `LANGGRAPH_STRICT_MSGPACK=false`, `JsonPlusSerializer(pickle_fallback=True)` runs `pickle.loads()` on checkpoint blobs from the `langgraph` Postgres schema. An attacker able to write to that schema (e.g., via a future injection elsewhere, or insider) plants a malicious pickle that executes on read.
- **Impact**: RCE in the api/worker process under the deserializing path, sharing `DATABASE_URL` credentials.
- **Remediation**: Keep the default and document that disabling strict msgpack is unsafe; consider removing the pickle-fallback option entirely.

[MEDIUM-003] Client-secret hashing uses fast HMAC-SHA256 instead of a password KDF

- **Severity**: MEDIUM
- **OWASP**: A04:2025 (Cryptographic Failures)
- **CWE**: CWE-916 (Use of Password Hash With Insufficient Computational Effort)
- **NIST CSF**: PR.DS, PR.AA
- **Compliance**: ASVS V2.4.1 | SOC2 CC6.1 | ISO A.8.24
- **Location**: app/api/routers/auth/secret_auth.py:122
- **Attack Vector**: `hash_secret()` = `HMAC-SHA256(pepper, f"{salt}:{secret}")` — billions of guesses/sec on GPU. If the `client_secrets` table is dumped and the pepper is recovered (it lives in an env var alongside DB creds, so a host compromise yields both), any client secret with $< \sim$128 bits of entropy is brute-forceable offline.
- **Impact**: Offline recovery of Mobile-API client secrets from a DB dump.
- **Remediation**: Hash client secrets with argon2id (already a project dependency for credential auth), optionally peppered. If secrets are guaranteed high-entropy random tokens, document that assumption and enforce minimum entropy at issuance.

[MEDIUM-004] Default credentials on Firecrawl Postgres and Grafana (opt-in profiles)

- **Severity**: MEDIUM
- **OWASP**: A02:2025 (Security Misconfiguration)
- **CWE**: CWE-1188 (Insecure Default Initialization) / CWE-1392 (Use of Default Credentials)
- **NIST CSF**: PR.AA, PR.PS
- **Compliance**: ASVS V2.1.1 | PCI DSS 8.3.6 | SOC2 CC6.1 | ISO A.8.9

- **Location**: ops/docker/docker-compose.yml:601,721 (FIRECRAWL_POSTGRES_PASSWORD:-postgres, FIRECRAWL_BULL_AUTH_KEY:-ratatoskr-local); :978 (GRAFANA_ADMIN_PASSWORD:-change-this-grafana-password
- **Attack Vector**: When the env vars are unset, these services start with well-known defaults. Firecrawl-Postgres is reachable by any container on the compose bridge network (postgres/postgres); Grafana admin is reachable by any host-local process on 127.0.0.1:3001.
- **Impact**: Firecrawl queue/DB takeover and scrape-result poisoning into the LLM pipeline; monitoring takeover and metric tampering. Mitigated by loopback binding and opt-in profiles, but a single compromised sidecar pivots through them.
- **Remediation**: Use ${VAR:?required} fail-fast for these like the main Postgres already does; never ship guessable default credentials.

[MEDIUM-005] MCP server has no per-tool rate limiting → LLM/scraper cost-DoS

- **Severity**: MEDIUM
- **OWASP**: A06:2025 (Insecure Design) / LLM10:2025 (Unbounded Consumption)
- **CWE**: CWE-770 (Allocation of Resources Without Limits or Throttling)
- **NIST CSF**: PR.DS, DE.CM
- **Compliance**: OWASP LLM Top 10 #10 | ASVS V11.1 | SOC2 CC9.1 | ISO A.12.1.3
- **Location**: app/mcp/tool_registrations.py:114-129; app/mcp/server.py:40-56
- **Attack Vector**: create_aggregation_bundle triggers a full scrape+LLM pipeline per submitted item with no MCP-layer rate limit (the Telegram UserRateLimiter 10/60s + 3-concurrent guard is not wired to the MCP transport). With auth_mode="disabled" (default stdio) there is no auth at all.
- **Impact**: A single MCP session can exhaust LLM budget and hammer external scrapers (cost-DoS).
- **Remediation**: Apply per-user/per-tool rate limiting and concurrency caps in the MCP layer; bound list sizes; require auth on any network-exposed MCP transport.

[MEDIUM-006] No PII redaction before scraped content is sent to external LLM providers

- **Severity**: MEDIUM
- **OWASP**: A04:2025 (Cryptographic/Data exposure) / LLM06:2025 (Sensitive Information Disclosure)
- **CWE**: CWE-359 (Exposure of Private Information)
- **NIST CSF**: PR.DS, GV.PO
- **Compliance**: GDPR Art. 25 | OWASP LLM Top 10 #6 | ASVS V8.3.4 | SOC2 CC6.7 | ISO A.8.11
- **Location**: app/application/services/summarization/graph_prompt.py:151; app/application/graphs/summari
- **Attack Vector**: clean_content_for_llm() strips boilerplate but is PII-unaware; full page text (names, emails, phone numbers, health/financial content) is transmitted to OpenRouter/OpenAI/Anthropic verbatim and processed per their retention policy.
- **Impact**: Data-minimization exposure / regulatory risk for any scraped page containing personal data.
- **Remediation**: Add an optional PII-redaction pass before LLM submission; document provider data-retention posture; allow operators to pin a zero-retention provider/route.

[MEDIUM-007] MCP unscoped mode + empty MCP_USER_ID default omits the per-user filter (cross-user data footgun)

- **Severity**: MEDIUM
- **OWASP**: A01:2025 (Broken Access Control)
- **CWE**: CWE-284 (Improper Access Control)
- **NIST CSF**: PR.AA, PR.DS
- **Compliance**: ASVS V4.1.1 | PCI DSS 7.1 | ATT&CK T1078 | SOC2 CC6.3 | ISO A.8.3
- **Location**: app/mcp/context.py:211-215; app/mcp/server.py:189-195; ops/docker/docker-compose.yml:408-41 (MCP_USER_ID:-, MCP_ALLOW_REMOTE_SSE=true)
- **Attack Vector**: When user_id is None (unscoped), request_scope_filters() drops the user_id predicate, so every MCP search/read tool scans all users' data. The compose mcp/mcp-write profiles default MCP_USER_ID empty with remote SSE enabled; the only guard is a startup warning. mcp-write additionally mounts /data read-write on 127.0.0.1:8201.

- **Impact**: A misconfigured/multi-user deployment serves cross-user data to any MCP client reaching the port. Intended owner behavior in single-tenant, but a sharp footgun.
- **Remediation**: Refuse to start unscoped SSE unless an explicit production override is set; require the JWT scope middleware for any network transport; surface a hard error (not a warning) when remote SSE is enabled without a user scope.

[MEDIUM-008] LLM debug payload logging exposes full prompts and `Authorization` header

- **Severity**: MEDIUM
- **OWASP**: A09:2025 (Security Logging Failures) / LLM06:2025
- **CWE**: CWE-532 (Insertion of Sensitive Information into Log File)
- **NIST CSF**: PR.DS, DE.AE
- **Compliance**: ASVS V7.1.1 | PCI DSS 10.3 | SOC2 CC7.2 | ISO A.8.11
- **Location**: `app/adapters/openrouter/chat_transport.py:399-405`
- **Attack Vector**: When `OPENROUTER_DEBUG_PAYLOADS=true`, `log_request_payload(payload.headers, payload.body, …)` writes the `Authorization: Bearer <KEY>` header and the full messages (system prompt + scraped PII) into structured logs.
- **Impact**: API-key and PII exposure to whatever consumes logs (Loki/ELK/SIEM) whenever debug payloads are enabled.
- **Remediation**: Redact the `Authorization` header and truncate/redact message bodies even under debug; gate this behind a non-production assertion and never log raw credentials.

[MEDIUM-009] Auth rate-limit bucket keyed on unvalidated `client_id` from the request body (bucket cycling)

- **Severity**: MEDIUM
- **OWASP**: A07:2025 (Authentication Failures)
- **CWE**: CWE-307 (Improper Restriction of Excessive Authentication Attempts)
- **NIST CSF**: PR.AA
- **Compliance**: ASVS V2.2.1 | PCI DSS 8.3.4 | ATT&CK T1110.003 | SOC2 CC6.1
- **Location**: `app/api/middleware.py:346-352, 678-681`
- **Attack Vector**: The auth bucket key is `f"client_id={client_id}|ip={client_ip}"` where `client_id` is read from the raw JSON body before validation. With `AUTH_ALLOW_ANY_CLIENT_ID=true` or an empty allowlist, rotating `client_id` per request mints a fresh counter each time → effectively unbounded attempts per IP. With an allowlist, the multiplier is bounded to `N_allowed × limit`.
- **Impact**: Brute-force amplification on the login/secret/refresh endpoints.
- **Remediation**: Validate `client_id` against the allowlist before it influences the rate-limit key, or key the bucket on IP (trusted, per HIGH-002) plus a server-trusted identifier only.

[MEDIUM-010] `/health/ready` returns raw DB exception strings to unauthenticated callers

- **Severity**: MEDIUM
- **OWASP**: A09:2025 / A10:2025
- **CWE**: CWE-209 (Error Message Containing Sensitive Information)
- **NIST CSF**: DE.AE, PR.IR
- **Compliance**: ASVS V7.4.2 | SOC2 CC6.1 | ISO A.12.4.1
- **Location**: `app/api/routers/health.py:125-135, 356-364`
- **Attack Vector**: The intentionally-unauthenticated readiness probe propagates `str(exc)` from a failed DB check into the response body, leaking asyncpg/SQLAlchemy host, port, DB name, and pool parameters.
- **Impact**: Infrastructure reconnaissance for any network-reachable unauthenticated caller. (`/health/detailed` is correctly auth-gated.)
- **Remediation**: Return a boolean `ready`/`unhealthy` status with no error detail to unauthenticated callers; log the detail server-side only.

[MEDIUM-011] **No `TrustedHostMiddleware` — Host-header injection**

- **Severity**: MEDIUM
- **OWASP**: A02:2025 (Security Misconfiguration)
- **CWE**: CWE-20 (Improper Input Validation)
- **NIST CSF**: PR.PS, DE.CM
- **Compliance**: ASVS V14.4.1 | SOC2 CC6.6 | ISO A.14.1.2
- **Location**: `app/api/main.py:229-254` (absent)
- **Attack Vector**: Without host validation, a poisoned `Host:` header can influence URL construction / origin comparisons and intermediate-cache keys, enabling password-reset/callback link poisoning or cache poisoning if any response is cached upstream.
- **Impact**: Host-header-based redirect/cache poisoning.
- **Remediation**: Add `TrustedHostMiddleware(allowed_hosts=[…])` with the production host-name(s).

[MEDIUM-012] **Unpinned sidecar/base images and lockfile-free sidecar dependency installs**

- **Severity**: MEDIUM
- **OWASP**: A03:2025 (Software Supply Chain Failures)
- **CWE**: CWE-494 / CWE-829
- **NIST CSF**: GV.SC, PR.DS
- **Compliance**: ASVS V10.3.2, V14.2.1 | SOC2 CC6.8 | ISO A.12.5.1
- **Location**: `ops/docker/defuddle/Dockerfile:21` + `defuddle/package.json:7-13` (caret ranges, no `package-lock.json`); `docker-compose.yml:588,641` (Firecrawl `:latest`); base images `python:3.13-slim`, `node:22-alpine`, `postgres:16-alpine`, `redis:7-alpine`, `prom/prometheus`, `grafana/grafana` pinned to tags not digests (`Dockerfile`, `Dockerfile.api`, compose). Note: `cloakbrowser` is correctly digest-pinned — use it as the pattern.
- **Attack Vector**: Mutable tags / live `npm install` resolve to attacker-controlled artifacts on a registry/account compromise; the defuddle sidecar renders content that flows into the LLM pipeline.
- **Impact**: Non-reproducible builds and supply-chain injection into scraping infrastructure.
- **Remediation**: Commit `package-lock.json` and use `npm ci`; pin Firecrawl and all base images to `@sha256:` digests with Dependabot/renovate to bump them.

[MEDIUM-013] **Monitoring containers lack `no-new-privileges` and mount sensitive host paths**

- **Severity**: MEDIUM
- **OWASP**: A02:2025 (Security Misconfiguration)
- **CWE**: CWE-250 (Execution with Unnecessary Privileges)
- **NIST CSF**: PR.PS, PR.AA
- **Compliance**: ASVS V14.2.7 | PCI DSS 2.2.4 | SOC2 CC6.3 | ISO A.8.9
- **Location**: `ops/docker/docker-compose.yml:1016-1030` (promtail), `:1031-1048` (node-exporter)
- **Attack Vector**: Neither sets `security_opt: [no-new-privileges:true]`. promtail mounts `/var/lib/docker/containers:ro` (every container's stdout/stderr, including secrets logged at startup); node-exporter mounts `/:/host/root:ro` (entire host FS). A compromise of either yields broad host read and setuid escalation potential.
- **Impact**: Secret leakage via container logs; host-filesystem read on container escape.
- **Remediation**: Add `no-new-privileges`, drop capabilities, and narrow the host mounts (these are the only core services lacking the hardening the app containers already have).

[MEDIUM-014] **Insufficient security logging/alerting on authentication events**

- **Severity**: MEDIUM
- **OWASP**: A09:2025 (Security Logging and Alerting Failures)
- **CWE**: CWE-778 (Insufficient Logging) / CWE-390
- **NIST CSF**: DE.AE, DE.CM, RS.AN

- **Compliance**: ASVS V7.2.1, V7.2.2 | PCI DSS 10.2 | SOC2 CC7.2 | ISO A.8.16
- **Location**: `app/api/middleware.py:78-80` (WebApp HMAC failures logged at DEBUG); `app/api/routers/auth/endpoints_credentials.py:115-134`, `endpoints_secret_keys.py:147-157` (no `source_ip` in failed-login logs); `app/api/middleware.py:36-37,426-443` (one-time Redis-unavailable warning, permanently suppressed thereafter)
- **Attack Vector**: Forged/replayed WebApp init-data failures are invisible at production INFO level; credential/secret-login failures omit source IP, making SIEM correlation and blocklisting impossible; sustained Redis degradation is logged once then silenced. No alerting exists on repeated auth failures (the A09:2025 emphasis).
- **Impact**: Brute-force / credential-stuffing and rate-limit degradation proceed without observable signal or alerting.
- **Remediation**: Log auth failures at WARNING with `source_ip` + `correlation_id`; emit a metric and alert on repeated failures and on Redis unavailability (re-arm the warning periodically rather than once-per-process).

[MEDIUM-015] Silent error swallowing in background tasks (digest unlock, github_sync vector init)

- **Severity**: MEDIUM
- **OWASP**: A10:2025 (Mishandling of Exceptional Conditions)
- **CWE**: CWE-390 (Detection of Error Condition Without Action)
- **NIST CSF**: DE.AE
- **Compliance**: ASVS V1.14.1 | SOC2 CC7.1 | ISO A.12.4.1
- **Location**: `app/tasks/digest.py:67-72` (`except Exception: pass` on lock release); `app/tasks/github_sync.py:10` (`except Exception: qdrant_store=None`)
- **Attack Vector**: A Redis error during digest lock release is swallowed silently, leaving the distributed lock held until TTL and skipping subsequent digest runs with no log/metric. A Qdrant init failure silently drops all GitHub embedding for the run.
- **Impact**: Silent functional/security-observability gaps; vector index drifts from source with no signal. (Note: the repo's `.bandit` blanket-skips B110, so these must be caught manually — see SMELL-001.)
- **Remediation**: Log `logger.exception(...)` and emit a metric in these handlers; never swallow exceptions without recording them.

Low & Informational Findings

[LOW-001] JWT legacy `aud`/`iss` grace window resets on every process restart

- **Severity**: LOW — **OWASP** A07:2025 — **CWE** CWE-613 — **NIST** PR.AA — **Compliance** ASVS V3.5.2 | SOC2 CC6.1
- **Location**: `app/api/routers/auth/tokens.py:95, 249-296`
- The 5-minute grace that accepts tokens missing `aud`/`iss` is anchored at module import time, so rolling/health-check restarts reopen it repeatedly. Signature + `exp` still enforced. **Remediation**: anchor the grace to a fixed migration timestamp (or remove it post-cutover).

[LOW-002] Magic-link verification token delivered in URL query string

- **Severity**: LOW — **OWASP** A07:2025 — **CWE** CWE-598 — **NIST** PR.DS — **Compliance** ASVS V3.7.1 | ISO A.8.5
- **Location**: `app/api/routers/auth/magic_link.py:69-98`
- One-time token in `GET …/verify?token=` lands in access logs, browser history, and `Referer`. **Remediation**: accept the token via POST body or exchange it immediately for a short-lived server-set credential; set `Referrer-Policy: no-referrer`.

[LOW-003] Blocking synchronous DNS (`socket.getaddrinfo`) on the async event loop

- **Severity**: LOW — **OWASP** A10:2025 (DoS amplification) — **CWE** CWE-400 — **NIST** PR.IR — **Compliance** ASVS V1.14 | SOC2 A1.1
- **Location**: `app/api/routers/proxy.py:71`; `app/api/routers/webhooks.py:135` → `app/domain/services/webhook`
- `is_url_safe()` calls blocking `getaddrinfo` inside async handlers; a slow-DNS host stalls the whole event loop (per redirect hop). SSRF protection itself is intact. **Remediation**: run the DNS check via `asyncio.to_thread` / async resolver with a short timeout.

[LOW-004] No global request-body size limit

- **Severity**: LOW — **OWASP** A06:2025 — **CWE** CWE-400 — **NIST** PR.IR — **Compliance** ASVS V13.2.1 | SOC2 CC6.1
- **Location**: `app/api/main.py` (no size middleware)
- The auth rate limiter reads `await request.body()` before counting, so a large body is buffered before any limit applies. **Remediation**: enforce a max body size at the proxy and/or a Starlette content-length middleware.

[LOW-005] Health endpoints are not rate-limited

- **Severity**: LOW — **OWASP** A06:2025 — **CWE** CWE-400 — **NIST** DE.CM — **Compliance** ASVS V4.10.2 | SOC2 CC6.1
- **Location**: `app/api/middleware.py:275-339`; `app/api/routers/health.py:336-382`
- `/health/ready` performs a real DB round-trip with no per-IP throttle; an unauthenticated flood can exhaust the asyncpg pool. **Remediation**: rate-limit health paths and/or cache readiness for a few seconds.

[LOW-006] Proxy endpoint logs raw user-supplied URL (may embed credentials)

- **Severity**: LOW — **OWASP** A09:2025 — **CWE** CWE-532 — **NIST** DE.AE — **Compliance** ASVS V7.1.1 | ISO A.8.11
- **Location**: `app/api/routers/proxy.py:72-73`
- `logger.warning("proxy_blocked_ssrf", extra={"url": current_url, …})` logs `https://user:pass@host/...` verbatim. **Remediation**: strip userinfo/query before logging URLs.

[LOW-007] Defuddle sidecar auth token empty by default (internal API unauthenticated)

- **Severity**: LOW — **OWASP** A02:2025 — **CWE** CWE-306 — **NIST** PR.AA — **Compliance** ASVS V4.1.2 | SOC2 CC6.1
- **Location**: `ops/docker/docker-compose.yml:73,175,335,777-778`
- `DEFUDDLE_AUTH_TOKEN:-` empty means any container on the compose network can call defuddle. **Remediation**: require a token (`${VAR:?}`) and rotate it.

[LOW-008] No wall-clock timeout wrapping the full summarize-graph invocation

- **Severity**: LOW — **OWASP** A04:2025/LLM10 — **CWE** CWE-400 — **NIST** PR.DS — **Compliance** ASVS V11.1.5 | SOC2 CC9.1
- **Location**: `app/application/graphs/summarize/state.py:49` (`MAX_REPAIR_ATTEMPTS=3`); `app/adapters/llm/base_client.py:84`
- Per-call timeout + recursion limit exist, but 1 summarize + 3 repair calls × 60s 240s per invocation with no deadline; concurrent large requests hold DB connections. **Remediation**: wrap `graph.ainvoke()` in `asyncio.timeout()`.

[LOW-009] `feedback_instructions` injected with a "Trusted" label

- **Severity**: LOW — **OWASP** A05:2025/LLM01 — **CWE** CWE-77 — **NIST** PR.DS — **Compliance** ASVS V5.2 | ISO A.14.1.2

- **Location**: app/application/services/summarization/graph_prompt.py:179-183
- If any user-influenced data ever populates feedback_instructions, the "Trusted correction instructions" label escalates it above the untrusted boundary. Currently app-controlled. **Remediation**: assert the source is app-only; never label user-derived content "trusted".

[LOW-010] Single Fernet key reused across three secret categories

- **Severity**: LOW — **OWASP** A04:2025 — **CWE** CWE-320 (Key Management) — **NIST** PR.DS — **Compliance** ASVS V6.4.1 | SOC2 CC6.7 | ISO A.8.24
- **Location**: app/db/models/repository.py:218; app/db/models/webwright.py:119; app/adapters/git_backup/mi
- GITHUB_TOKEN_ENCRYPTION_KEY encrypts GitHub tokens, Webwright per-domain cookies, and git-clone creds with no context separation; one key compromise exposes all three and rotation is coupled. **Remediation**: derive category-specific subkeys via HKDF from the master key.

[LOW-011] Cleartext transport to Qdrant and Postgres when remote (no TLS configured)

- **Severity**: LOW — **OWASP** A04:2025 — **CWE** CWE-319 (Cleartext Transmission) — **NIST** PR.DS — **Compliance** ASVS V9.1.1 | SOC2 CC6.7 | ISO A.8.24
- **Location**: app/config/integrations.py:458 / app/infrastructure/vector/qdrant_store.py:175 (default http://); app/db/session.py:68 (no ssl in asyncpg connect_args)
- Fine for loopback, but moving Qdrant/Postgres to another host exposes embeddings, metadata, refresh tokens, and Fernet ciphertext to passive interception. **Remediation**: add a field_validator rejecting http:// Qdrant and a DATABASE_SSL option for asyncpg in non-local environments.

[LOW-012] Access tokens not revoked on logout (JTI generated but never checked)

- **Severity**: LOW — **OWASP** A07:2025 — **CWE** CWE-613 — **NIST** PR.AA — **Compliance** ASVS V3.3.3 | SOC2 CC6.3 | ISO A.8.5
- **Location**: app/api/routers/auth/tokens.py:149-156; app/api/routers/auth/endpoints_sessions.py:252-281
- Logout revokes only the refresh token; the access token (30 min) stays valid via its jti, which is never persisted/checked. **Remediation**: maintain a short-TTL JTI denylist (Redis) checked on each request, or shorten access-token TTL.

[INFO-001] Telethon session file (ratatoskr_bot.session) stored unencrypted on disk

- **Severity**: INFO — **OWASP** A04:2025 — **CWE** CWE-312 — **NIST** PR.DS — **Compliance** SOC2 CC6.7 | ISO A.8.24
- Correctly .gitignored and .dockerignored (verified: never committed). The MTProto auth key grants full Telegram access independent of BOT_TOKEN and survives token rotation. **Remediation**: chmod 600, store on an encrypted volume, and revoke/regenerate if the host is ever compromised. Guard against accidental git add -A.

[INFO-002] Secret-scan (gitleaks) — 18 matches, all false positives

- **Severity**: INFO — **OWASP** A04:2025 — **CWE** CWE-798 — **NIST** ID.AM — **Compliance** ISO A.8.11
- 2845 commits scanned. All 18 hits are non-secrets: env-var references ("${SAFETY_API_KEY}:" in two workflows), repeated-hex placeholders (JWT_SECRET = "0123456789abcdef0123456789abcdef" in scripts/migration/run_m8_*), doc/tutorial example JWTs/curl headers, test fixtures (tests/api/test_secret_login.py, tests/test_allowlist_unification.py), and a throw-away RSA key in a Rust test (rust/crates/bsr-mobile-api/tests/core_domain_routes.rs). No live production secret leaked. **Remediation**: optionally relocate the Rust test key to a generated fixture and extend the gitleaks allowlist for the migration-script placeholders.

[INFO-003] OpenAPI `servers:` and `/api` payload disclose production hostname/docs path

- **Severity**: INFO — **OWASP** A02:2025 — **CWE** CWE-200 — **NIST** PR.PS — **Compliance** ISO A.8.9
- `app/api/main.py:194-197` hardcodes `https://ratatoskrapi.po4yka.com`; `:347-355` returns `"docs": "/docs"` regardless of `API_DOCS_ENABLED`. Benign (docs 404 when disabled). **Remediation**: derive the server URL from config; omit the docs path when disabled.

[INFO-004] Local dependency scan could not run; CI dependency scanning is in place

- **Severity**: INFO — **OWASP** A03:2025 — **CWE** CWE-1104 — **NIST** GV.SC, ID.RA — **Compliance** ISO A.8.8 | SOC2 CC7.1
- Local `pip-audit` aborted building its ephemeral venv (`ensurepip` SIGABRT — sandbox/toolchain issue, not a code defect). The repo's CI runs Bandit, pip-audit, Safety, OSV, Gitleaks and `cargo deny`; `osv-scanner.toml` documents one accepted exception (torch DoS `GHSA-rrmf-rvhw-rf47`, ml-extra only, not in the request path). **Remediation**: re-run `pip-audit`/`osv-scanner` in CI's environment to confirm zero unaccepted CVEs; keep the dependency-review gate blocking (see HIGH-007).

Gray-Box Findings

[GRAY-001] Rate-limit cap multiplies behind a proxy / across workers

- **Severity**: HIGH — **OWASP** A07:2025 — **CWE** CWE-348 — **NIST** PR.AA — **Compliance** ASVS V4.10.3 | SOC2 CC6.1
- **Tested As**: unauthenticated attacker against the Mobile API
- **Endpoint**: `POST /v1/auth/secret-login`, `POST /v1/auth/credentials-login`, `POST /v1/auth/refresh`
- **Expected**: N attempts/window/source, enforced globally
- **Actual**: cap is per-proxy-IP (all clients share one bucket) and per-process when Redis is down, and per rotated `client_id` body value. Cross-references HIGH-001, HIGH-002, MEDIUM-009.
- **Remediation**: see HIGH-001/HIGH-002/MEDIUM-009.

[GRAY-002] Unauthenticated readiness probe leaks DB error detail

- **Severity**: MEDIUM — **OWASP** A10:2025 — **CWE** CWE-209 — **NIST** DE.AE
- **Tested As**: unauthenticated network-reachable client
- **Endpoint**: `GET /health/ready`
- **Expected**: opaque ready/not-ready
- **Actual**: returns `str(exc)` with DB host/schema on failure. See MEDIUM-010.

[GRAY-003] IDOR guard relies on application layer at some repository methods

- **Severity**: LOW — **OWASP** A01:2025 — **CWE** CWE-639 — **NIST** PR.AA — **Compliance** SOC2 CC6.3
- **Tested As**: authenticated user probing object-level access
- **Endpoint**: refresh-token revoke / client-secret expiry paths
- **Expected**: DB-level `user_id` predicate on every mutation (project's own rule)
- **Actual**: `async_revoke_refresh_token` (`auth_repository.py:213-233`) has no `user_id` predicate, and `check_expired()` (`secret_auth.py:186-192`) can pass `owner_user_id=None`. Not currently reachable without a prior ownership check, but violates the documented defense-in-depth IDOR rule. Session-by-id revoke and list paths *are* correctly scoped.
- **Remediation**: add the `user_id`/`owner_user_id` predicate at the DB layer on these two methods.

[GRAY-004] MCP tools enumerate all users' data in unscoped mode

- **Severity**: MEDIUM — **OWASP** A01:2025 — **CWE** CWE-284 — **NIST** PR.AA

- **Tested As**: MCP client against an unscoped SSE deployment
- **Endpoint**: MCP `search_articles`, `x_search`, `semantic_search`, etc.
- **Expected**: results scoped to the caller
- **Actual**: with `MCP_USER_ID` empty + remote SSE, no `user_id` filter is applied. See MEDIUM-007.

[GRAY-005] No alerting on repeated auth failures / rate-limit degradation

- **Severity**: MEDIUM — **OWASP** A09:2025 — **CWE** CWE-778 — **NIST** DE.AE, RS.MA
- **Tested As**: defender reviewing detection coverage
- **Endpoint**: auth endpoints + middleware
- **Expected**: alert on repeated failures and on Redis-unavailable degradation
- **Actual**: failures logged (some at DEBUG, some without IP) but no alerting; Redis-down warning fires once. See MEDIUM-014.

Security Hotspots

[HOTSPOT-001] SSRF transport (`app/security/ssrf.py`) — load-bearing, IP-pinning

- **OWASP** A01:2025 — **CWE** CWE-918 — **NIST** PR.DS — **Location** `app/security/ssrf.py`
- `SafeAsyncTransport`/`SafeSyncTransport` resolve DNS, check every IP against `BLOCKED_NETWORKS` (incl. 169.254.0.0/16, RFC1918, loopback, link-local, CGNAT, IPv4-mapped IPv6), then rewrite the URL to the resolved IP — closing TOCTOU for all direct-httpx callers. **Review guidance**: any new outbound HTTP MUST go through `make_safe_async_client`/`make_safe_sync_client`; never instantiate a bare `httpx.AsyncClient`. The non-httpx clients (crawlee/Playwright/git) bypass this — see MEDIUM-001.

[HOTSPOT-002] Auth fail-closed defaults — `fail_open_when_empty=False`

- **OWASP** A07:2025 — **CWE** CWE-280 — **NIST** PR.AA — **Location** `app/api/dependencies.py:115`, `app/api/routers/auth/webapp_auth.py:104`, `app/adapters/telegram/access_controller.py:51-53`
- Deny-by-default when `ALLOWED_USER_IDS` is empty; the Telegram controller raises at boot if empty. **Review guidance**: never flip these to fail-open; the boot-time raise is intentional.

[HOTSPOT-003] Constant-time secret comparison

- **OWASP** A07:2025 — **CWE** CWE-208 — **NIST** PR.AA — **Location** `tokens.py:145`, `webapp_auth.py:69`, `endpoints_telegram.py:151`, `secret_auth.py:126`
- All secret/HMAC/nonce comparisons use `hmac.compare_digest`. **Review guidance**: never replace with `==`.

[HOTSPOT-004] Telegram HTML output escaping is field-by-field

- **OWASP** A05:2025 — **CWE** CWE-79 — **NIST** PR.DS — **Location** `app/adapters/telegram/callback_action_pre`
- Every LLM-generated field is `html.escape()`d before `parse_mode="HTML"`, but per-field — a new contract field rendered without an explicit escape silently introduces XSS. **Review guidance**: add a CI test asserting escape coverage for every field rendered in HTML mode.

[HOTSPOT-005] RAG scope uses `user_scope + environment`, not `user_id`

- **OWASP** A01:2025 — **CWE** CWE-284 — **NIST** PR.AA — **Location** `app/application/graphs/summarize/nodes/g`
- Qdrant points carry no `user_id`; correct for single-tenant, but multi-tenancy without adding `user_id` to the point payload AND the filter would cross-pollinate RAG context. **Review guidance**: gate multi-tenancy behind a vector re-index + filter change (record an ADR).

[**HOTSPOT-006**] **Webwright double-gating (feature flag + non-empty host allowlist)**

- **OWASP** A01:2025 — **CWE** CWE-918 — **NIST** PR.AA — **Location** `app/adapters/content/scraper/factory.py:`
- Empty allowlist short-circuits provider construction (no fail-open). **Review guidance**: keep both gates; never construct the provider with an empty allowlist.

[**HOTSPOT-007**] **Git-backup path-traversal guard**

- **OWASP** A01:2025 — **CWE** CWE-22 — **NIST** PR.DS — **Location** `app/adapters/git_backup/mirror_service.py` (`_mirror_destination`, `_assert_inside_data_path`)
- Mirror name sanitized (`\x00`, `/`, `..` → `_`) then `resolve()`d and asserted under `data_path`; credentials path validated against `^[A-Za-z0-9_./()\-]+$`. **Review guidance**: keep both the sanitize and the resolved-prefix assertion.

[**HOTSPOT-008**] **Per-process in-memory state used for security counters**

- **OWASP** A06:2025 — **CWE** CWE-799 — **NIST** PR.AA — **Location** `app/adapters/telegram/access_controller.` `app/api/middleware.py` local limiter
- Brute-force/rate counters in per-process dicts diverge across workers/pods. **Review guidance**: back security counters with Redis in any horizontally-scaled deployment.

Code Smells

[**SMELL-001**] **Project-wide Bandit B110 suppression hides silent-failure bugs**

- **OWASP** A10:2025 — **CWE** CWE-390 — **Location** `.bandit` (`skips = B110`)
- Blanket-skipping `try_except_pass` means the static scanner can never flag the silent swallows in MEDIUM-015. **Suggestion**: remove the global skip; use narrow inline `# nosec B110` with a justification only where intentional.

[**SMELL-002**] **Production/debug behavior keyed on log level, not environment**

- **OWASP** A10:2025 — **CWE** CWE-209 — **Location** `app/api/error_handlers.py:144-147`
- Conflates verbosity with security posture (root of HIGH-009). **Suggestion**: branch on `APP_ENV`.

[**SMELL-003**] **Advisory-only security control (prompt-injection detector)**

- **OWASP** A06:2025 — **CWE** CWE-20 — **Location** `app/core/content_cleaner.py:24-56`
- A control that only sets a metadata flag reads as protection but enforces nothing (root of HIGH-008). **Suggestion**: either enforce or clearly document it as telemetry, not a control.

[**SMELL-004**] **Security counters in per-process memory**

- **OWASP** A06:2025 — **CWE** CWE-799 — **Location** `access_controller.py:56-59`, middleware local limiter
- See HOTSPOT-008. **Suggestion**: centralize in Redis.

[**SMELL-005**] **Field-by-field output escaping with no escape-by-default wrapper**

- **OWASP** A05:2025 — **CWE** CWE-79 — **Location** `callback_action_presenters.py`
- See HOTSPOT-004. **Suggestion**: a single escaping serializer for HTML-mode messages + a CI guard.

[SMELL-006] Guessable default credentials embedded as compose fallbacks

- **OWASP** A02:2025 — **CWE** CWE-1188 — **Location** `docker-compose.yml` (Firecrawl, Grafana, Defuddle)
- Mixing `${VAR:?required}` (main Postgres — good) and `${VAR:-guessable}` (sidecars — bad) is inconsistent. **Suggestion**: standardize on fail-fast `:?` for all secret-bearing variables.

Recommendations Summary

Priority 1 (do first — production-exposure blockers): 1. **Rate limiting** — make Redis required (fail-closed) for auth buckets, trust `X-Forwarded-For` only from configured proxies, and validate `client_id` before it keys a bucket (HIGH-001, HIGH-002, MEDIUM-009). 2. **Security headers + TrustedHost** — add CSP/HSTS/XFO/XCTO/Referrer/Permissions headers and `TrustedHostMiddleware` (HIGH-003, MEDIUM-011). 3. **Error handling** — decouple verbose errors from log level; opaque health-probe errors (HIGH-009, MEDIUM-010).

Priority 2 (supply chain — A03/A08): 4. Pin GitHub Actions to SHAs, fix `UV_INDEX_STRATEGY`, restore the dependency-review gate, pin the Webwright ref and all images to digests, commit the defuddle lockfile (HIGH-004 through HIGH-007, MEDIUM-012).

Priority 3 (LLM pipeline — A05): 5. Apply the untrusted-content boundary on every LLM path; move RAG grounding out of the system prompt and sanitize it; stop treating injection detection as a control (HIGH-008, HIGH-010, LOW-009).

Priority 4 (defense-in-depth & observability): 6. argon2id for client secrets, HKDF-separated Fernet subkeys, MCP rate-limiting + scope enforcement, PII redaction before LLM, log auth failures with IP + alerting, remove silent swallows and the blanket B110 skip (MEDIUM-003/005/006/007/008/014/015, LOW-010).

Methodology

Aspect	Details
Phases executed	1-5 (reconnaissance, white-box, gray-box, hotspots, code smells)
Frameworks detected	FastAPI (primary), Telethon (Telegram), SQLAlchemy 2.0 + asyncpg, Taskiq, LangGraph, MCP, Qdrant; React SPA served as static assets
White-box categories	All 20 attack categories; deepest on A01/A02/A03/A04/A05/A07/A08/A09/A10 and AI/LLM
Gray-box testing	Auth/rate-limit boundaries, IDOR object-level checks, health/MCP exposure, error differentials
Security hotspots	8 (crypto, auth boundaries, SSRF transport, output encoding, RAG scope, path traversal, per-process state)
Code smells	structural, data-handling, error-handling, dependencies, design (6)
Tooling	gitleaks (2845 commits, 18 matches all false-positive), bandit (`-ll`, B110 skipped), pandoc (PDF); pip-audit failed locally (sandbox venv), CI covers dependency scanning
Packs loaded	none

Aspect	Details
Scope exclusions	none (no `.security-audit-ignore`)
Baseline comparison	none (no `.security-audit-baseline.json`)
OWASP Top 10:2025	10/10 categories covered
NIST CSF 2.0	GV, ID, PR, DE, RS covered; RC out of scope
CWE	23 unique CWE IDs identified
SANS/CWE Top 25	6/25 matched
ASVS 5.0	V1, V2, V4, V5, V6, V7, V8, V11, V14
Additional frameworks	PCI DSS 4.0.1, MITRE ATT&CK, SOC 2, ISO 27001:2022

Report generated by Claude Security Audit